



PDF Download
3658644.3691377.pdf
06 April 2026
Total Citations: 1
Total Downloads: 2863

 Latest updates: <https://dl.acm.org/doi/10.1145/3658644.3691377>

POSTER

Poster: Automated Dependency Mapping for Web API Security Testing Using Large Language Models

WANPENG LI, University of Aberdeen, Aberdeen, Scotland, U.K.

YUEJUN GUO, Luxembourg Institute of Science and Technology, Esch-sur-Alzette, Esch-sur-Alzette, Luxembourg

Open Access Support provided by:

Luxembourg Institute of Science and Technology

University of Aberdeen

Published: 09 December 2024

[Citation in BibTeX format](#)

CCS '24: ACM SIGSAC Conference on
Computer and Communications Security
October 14 - 18, 2024
UT, Salt Lake City, USA

Conference Sponsors:
SIGSAC

Poster: Automated Dependency Mapping for Web API Security Testing Using Large Language Models

Wanpeng Li*
wangpeng.li@abdn.ac.uk
University of Aberdeen
Aberdeen, UK

Yuejun Guo*
yuejun.guo@list.lu
Luxembourg Institute of Science and Technology
Esch-sur-Alzette, Luxembourg

Abstract

Dependency extraction is crucial in web API security testing, as it helps identify the required API sequences to exploit a vulnerability. Traditional methods are generally rule-based and require extensive manual analysis of API specification documents by domain experts to formulate appropriate rules. This manual process is not only time-consuming and labor-intensive but also prone to missing dependencies and inaccuracies, which can compromise the effectiveness of security testing. In this paper, we explore the potential of large language models (LLMs) to automate dependency mapping in web APIs. By leveraging the capabilities of advanced LLMs such as GPT-3.5, Mistral-7B-Instruct, and Llama-3-8B-Instruct, which include understanding and generating natural language, we aim to streamline the dependency mapping process, reducing the need for manual analysis and enhancing accuracy. Our preliminary experiments demonstrate that this approach can effectively build dependency mappings, offering a promising alternative to traditional rule-based approaches.

CCS Concepts

• **Security and privacy** → **Web application security**; • **Computing methodologies** → **Artificial intelligence**; *Information extraction*.

Keywords

Dependency mapping, API security testing, Large language model

ACM Reference Format:

Wanpeng Li and Yuejun Guo. 2024. Poster: Automated Dependency Mapping for Web API Security Testing Using Large Language Models. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24)*, October 14–18, 2024, Salt Lake City, UT, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3658644.3691377>

1 Introduction

The web API ecosystem has experienced significant growth in recent years, with approximately 1.29 billion API requests¹ generated in the past year alone. Ensuring the security of web APIs is crucial.

*Both authors contributed equally to this research.

¹<https://www.postman.com/state-of-api/>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

CCS '24, October 14–18, 2024, Salt Lake City, UT, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0636-3/24/10

<https://doi.org/10.1145/3658644.3691377>

Existing security testing methods primarily depend on detecting “500 Internal Server Errors” to identify bugs within API interfaces. One of the main challenges in testing web API security is determining the test case sequences (API dependencies) that can trigger vulnerable functions in the API.

Web API security testing typically involves four steps. The first step is collecting the OpenAPI Specification (OAS) that describes the web APIs being tested. The OAS provides valuable information, such as API service endpoints (paths), CRUD operations (GET, POST, PUT, DELETE) for each endpoint, input parameters, and expected responses. This specification helps generate valid API operations and facilitates communication with API services through HTTP requests. The second step involves generating the API sequences (or API dependencies), which reflect the state transitions of the API under test. This step is essential because some vulnerabilities require multiple steps to exploit, meaning that simply injecting an attack payload into a single API request may not trigger the vulnerability [1]. The generated sequence of API requests simulates realistic usage patterns and interactions, ensuring that the security testing covers complex scenarios where multiple API calls interact. The third step involves generating attack payloads for the API under test based on the API sequences generated in step 2. This step is crucial because different vulnerabilities are triggered by different payloads, making payload generation a critical aspect of security testing. By carefully crafting and injecting various payloads, testers can simulate a wide range of attack vectors [2], including SQL injection, cross-site scripting (XSS), and other common exploits. Finally, in the fourth step, the API sequences from step 2 and the payloads generated in step 3 are combined to create comprehensive test cases. This step involves executing these test cases and meticulously evaluating the responses from the API under test. The evaluation process is crucial and extends beyond simply checking for 5xx status codes, as many vulnerabilities do not trigger such error responses. In this paper, we focus on the second step given its importance.

Recently, researchers [1, 2] have developed tools to test web API vulnerabilities by utilizing custom algorithms to create test cases. While these tools have demonstrated good performance, they rely on rule-based algorithms to extract API dependencies, which presents several limitations. Firstly, significant effort from human experts [1] is required to check OAS documentation for APIs under test and build the necessary rules for creating API sequences. This approach might be feasible for small-scale OAS documentation, but as the number of web API endpoints increases, the time and effort required to build these rules escalate substantially. Secondly, different researchers [1, 2] use varied rules to construct their own API sequences, leading to inconsistencies. These rules often fall

short in identifying all possible API dependencies, resulting in gaps in the coverage. Lastly, complex API dependencies may be overlooked by rule-based approaches, further leading to insufficient coverage of API sequences.

Nowadays, the application of large language models (LLMs) has emerged as a promising alternative to traditional rule-based approaches in various tasks such as software vulnerability detection [7] and code generation [6], as they excel in understanding and processing complex logical relationships and reasoning. By leveraging its inherent capability to automatically discern intricate patterns and features from historical data that are often elusive to human detection, LLM has demonstrated its ability to automate these tasks with remarkable performance, requiring minimal human intervention. Given this, we explore the use of LLMs, in this paper, to automatically extract dependencies between APIs that can be further used for security testing. We have developed a prototype that extracts dependencies from a given OAS document. Through our experiments with advanced LLMs, such as GPT-3.5, Mistral-7B-Instruct, and Llama-3-8B-Instruct, we demonstrate the effectiveness of this approach, providing a more efficient and reliable alternative to traditional rule-based tools.

2 LLM-driven API dependency mapping

Figure 1 illustrates our methodology of leveraging LLMs to build dependency mapping, which encompasses three key components:

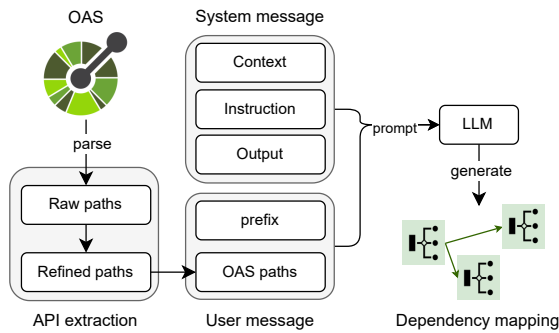


Figure 1: Overview of LLM-based API dependency mapping.

API extraction. Given an OAS document, this first step involves extracting the raw paths that define various endpoints. Following this, the raw paths are refined by extracting key information such as HTTP methods, parameters, and summary. The path refinement is crucial for several reasons. First, LLMs have inherent limitations on the length of data that can process at once, making it impractical to handle an entire OAS document directly. This is particular important when the document is long and the computing resource is limited. Secondly, truncating an OAS document into multiple parts to fit within these limits will lose essential contextual information, which can lead to missing dependencies. Thirdly, by refining paths, we ensure that the LLM focuses on the most relevant features, which will improve the accuracy of extracted dependencies.

Prompt. Prompt is the crucial component in the entire methodology as it provides the context and guides the model's output, ensuring the generation is relevant and accurate. The prompt is carefully structured to consist of a system message and a user message as shown below. The system message proves the context and set the goal of extracting dependencies from the OAS document. It provides instructions and guide the LLM to focus on three different dependencies, including token dependencies, data dependencies, and parameter dependencies, by giving clear definitions. In addition, the system message formulates the output format for the dependency mapping, ensuring that the LLM produces results in a structured manner. On the other hand, the user message provides specific details related to the refined paths extracted in the first component.

Prompt

System message:

The OpenAPI Specification (OAS) document describes a set of API endpoints. Analyze the dependencies among various API endpoints described in the OAS to build a dependency graph. The goal is to understand how API calls are interconnected based on their parameters, tokens, and data dependencies.

Focus only on the paths and methods provided in the document—do not introduce any new endpoints. The analysis should identify and explain the dependencies among these API calls.

Instructions:

1. Token Dependencies:

- (1) Identify which of the given API endpoints require authentication tokens.
- (2) Determine the dependencies where tokens from one endpoint are necessary for another endpoint.

2. Data Dependencies:

- (1) Determine if any of the given API endpoints rely on data or results from other endpoints listed in the document.

3. Parameter Dependencies:

- (1) Analyze the parameters used by each given API endpoint.
- (2) Determine if parameters from one endpoint are required by or influence other endpoints.

Dependency Mapping:

- (1) Provide a comprehensive mapping of dependencies among the API endpoints.

- (2) Use the format A -> B (Type), where:

- A is the source endpoint.

- B is the target endpoint.

- Type indicates the nature of the dependency (Token, Data, or Parameter).

Output:

A detailed dependency mapping among the given API endpoints in the format A -> B (Type), where each dependency is categorized by its type (Token, Data, or Parameter).

User message:

Here are paths extracted from the OAS document for analysis:

<extracted paths>

LLM. The LLM serves as the driving force behind our methodology. Advanced LLMs such as GPT-3.5 [5] and Llama-3 [3] can be employed to process the prompt and generate the dependency mapping.

3 Preliminary results

In this study, we utilize OAS documents from two sources: OWASP Pixi², which is designed with deliberately vulnerable APIs, and Appwrite³, a medium-scale API application [2]. The OWASP Pixi OAS document includes 21 APIs and Appwrite has 65. Regarding the LLM, we employ three models: OpenAI’s GPT-3.5 [5], Meta’s Llama-3-8B-Instruct [3], and Mistral AI’s Mistral-7B-Instruct [4]. Table 1 shows the dependency mapping results using these LLMs.

Table 1: Comparison results of dependency mapping by different LLMs. Number: the amount of extracted dependencies. Accuracy: the correct percentage of extracted dependencies. Miss: the number of dependencies that are missed by a certain LLM.

LLM	OWASP Pixi			Appwrite		
	Number	Accuracy	Miss	Number	Accuracy	Miss
GPT-3.5	57	100%	0	16	100%	9
Llama-3	9	22%	55	11	0%	25
Mistral	23	74%	40	19	11%	23

We manually evaluated the API dependencies generated by various LLMs. GPT-3.5 demonstrated outstanding performance, achieving 100% accuracy. Notably, the Pixi app has a vulnerability at the `/api/user/edit_info` endpoint. This vulnerability allows an attacker who knows the victim’s username to update the victim’s information, as the endpoint is not protected by authentication. GPT-3.5 successfully identified this issue and did not generate an `/api/login -> /api/user/edit_info` dependency. In contrast, Llama-3-8B-Instruct struggled significantly, identifying only nine API dependencies and just two token dependencies out of 36 available in the Pixi app. It also had difficulty recognizing API sequences that depend on data and parameter dependencies. On the other hand, Mistral-7B-Instruct generated 23 API dependencies and successfully identified 17 token dependencies out of 36. However, it failed to correctly identify data and parameter dependencies.

Appwrite offers 65 APIs and its OpenAPI Specification (OAS) provides more detailed information than Pixi’s, such as security schemas. This requires LLMs to thoroughly understand the OAS document to generate accurate API dependencies. GPT-3.5 demonstrated 100% accuracy in generating 16 API dependencies but failed to detect nine others (Figure 2). Of these nine, four were missed because GPT-3.5 focused on the session-related parameter dependencies within the API group. It correctly identified dependencies such as `/user/{userId} -> /user/{userId}/sessions`, but missed others like `/user/{userId} -> /user/{userId}/logs` and `/user/userId -> /user/userId/prefs`. Additionally, another four undetected dependencies were related to the `/storage/files` API group. GPT-3.5 failed to identify any parameter dependencies, such as `/storage/files -> /storage/files/{fileId}`. A manual review suggests that this oversight might be due to the API description, which states, “Get a list of all the user files,” instead of referencing `fileId`. Furthermore, GPT-3.5 did not detect the dependency `/account/sessions -> /avatars/browsers/{code}`, which is deeply embedded in the description of the `/avatars/brow`

²<https://github.com/DevSlop/Pixi>

³<https://appwrite.io/>

```

Summary of Dependency Mapping
Token Dependencies:
No specific token dependencies between endpoints in this example.
Data Dependencies:
/account/recovery (POST) -> /account/recovery (PUT) (Data)
/account/verification (POST) -> /account/verification (PUT) (Data)
Parameter Dependencies:
/database/collections (POST) -> /database/collections/collectionId (GET, PUT, DELETE) (Parameter)
/database/collections/collectionId (GET) -> /database/collections/collectionId/documents (GET, POST) (Parameter)
/database/collections/collectionId/documents (POST) -> /database/collections/collectionId/documents/documentId (GET, PATCH, DELETE) (Parameter)
/functions (POST) -> /functions/functionId (GET, PUT, DELETE) (Parameter)
/functions/functionId (GET) -> /functions/functionId/execution (GET, POST) (Parameter)
/functions/functionId/execution (POST) -> /functions/functionId/executionId (GET) (Parameter)
/functions/functionId (GET) -> /functions/functionId/tags (GET, POST) (Parameter)
/functions/functionId/tags (POST) -> /functions/functionId/tags/tagId (GET, DELETE) (Parameter)
/teams (POST) -> /teams/teamId (GET, PUT, DELETE) (Parameter)
/teams/teamId (GET) -> /teams/teamId/memberships (GET, POST) (Parameter)
/teams/teamId/memberships (POST) -> /teams/teamId/memberships/membershipId (DELETE, PATCH) (Parameter)
/users (POST) -> /users/userId (GET, DELETE) (Parameter)
/users/userId (GET) -> /users/userId/sessions (GET, DELETE) (Parameter)
/users/userId/sessions (DELETE) -> /users/userId/sessions/sessionId (DELETE) (Parameter)

```

Figure 2: Dependency mapping by GPT-3.5 on Appwrite.

ers/code API: “The code argument receives the browser code as it appears in your user `/account/sessions` endpoint”. In comparison, Llama-3-8B-Instruct failed to generate any useful API dependencies, while Mistral-7B-Instruct identified 19 APIs, but only two were correct.

4 Conclusion and future work

Our study demonstrates the capability of LLMs in extracting dependencies between APIs from the OpenAPI Specification (OAS). Through experiments on three state-of-the-art LLMs, OpenAI’s GPT-3.5 [5], Meta’s Llama-3-8B-Instruct [3], and Mistral AI’s Mistral-7B-Instruct [4], and two test cases on OWASP Pixi and Appwrite, we observed that GPT-3.5 excels in extracting dependencies, it requires improvements in capturing more complex API relations. Llama-3-8B-Instruct and Mistral-7B-Instruct have room for growth in processing and understanding comprehensive API interactions.

Future research will be dedicated to the API extraction from OAS to ensure the refined paths cover all necessary information while remaining within the processing capacity limits of LLMs.

Acknowledgments

Yuejun Guo is supported by the European Commission under the Horizon Europe Programme, as part of the project LAZARUS (Grant Agreement no. 101070303) (<https://lazarus-he.eu/>). The content of this article does not reflect the official opinion of the European Union. Responsibility for the information and views expressed therein lies entirely with the authors.

References

- [1] Gelei Deng, Zhiyi Zhang, Yuekang Li, Yi Liu, Tianwei Zhang, Yang Liu, Guo Yu, and Dongjin Wang. 2023. NAUTILUS: Automated RESTful API Vulnerability Detection. In *32nd USENIX Security Symposium (USENIX Security 23)*. 5593–5609.
- [2] Wenlong Du, Jian Li, Yanhao Wang, Libo Chen, Ruijie Zhao, Junmin Zhu, Zheng-guang Han, Yijun Wang, and Zhi Xue. 2024. Vulnerability-oriented Testing for RESTful APIs. In *33rd USENIX Security Symposium (USENIX Security 24)*.
- [3] Meta. 2024. Meta-Llama-3-8B-Instruct on Hugging Face. Retrieved July 26, 2024 from <https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct>
- [4] Mistral AI. 2023. Mistral-7B-Instruct-v0.2 on Hugging Face. Retrieved July 26, 2024 from <https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.2>
- [5] OpenAI. 2022. ChatGPT (GPT3.5). Retrieved July 26, 2024 from <https://openai.com/index/chatgpt/>
- [6] Baptiste Rozière, Jonas Gehring, and others. 2024. Code llama: open foundation models for code. arXiv:2308.12950
- [7] Xin Zhou, Ting Zhang, and David Lo. 2024. Large language model for vulnerability detection: emerging results and future directions. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results* (Lisbon, Portugal) (ICSE-NIER’24). Association for Computing Machinery, New York, NY, USA, 47–51. <https://doi.org/10.1145/3639476.3639762>